Quantum Testing Agent

Agente de pruebas cuánticas aislado en sandbox Docker
para Vultr: Agent Rush Hackathon

Jesús León Porras
jalp17@github

8 de noviembre de 2026

Abstract

Este documento presenta el diseño, arquitectura y plan de implementación del **Quantum Testing Agent**, un agente web que ejecuta pruebas de circuitos y simulaciones cuánticas dentro de un sandbox Docker aislado en Vultr. El proyecto participa en el desafío *Blast Radius Zero* del hackathon Vultr: Agent Rush (lablab.ai × Vultr, 3–8 nov 2026). El agente planifica la prueba mediante un LLM (Vultr Serverless Inference), genera código en el framework solicitado (Qiskit, PennyLane, Cirq, Azure Quantum), lo ejecuta en un contenedor desechable con límites de recursos y devuelve un resultado verificable. La contención es obligatoria: el sandbox absorbe fallos (bucles infinitos, comandos destructivos) sin comprometer el host ni el backend.

Contents

1 Contexto del hackathon **2**
1.1 Objetivo . . . 2
1.2 Desafío elegido: Blast Radius Zero . . . 2
1.3 Stack tecnológico requerido . . . 2
1.4 Entregables requeridos . . . 2
1.5 Criterios de juzgamiento (1–5 cada uno) . . . 3

2 Descripción del proyecto **3**
2.1 Qué es . . . 3
2.2 Problema . . . 3
2.3 Solución . . . 3
2.4 Compatibilidad . . . 3

3 Arquitectura **3**
3.1 Modelo C4 – Nivel 1: Contexto . . . 3
3.2 Modelo C4 – Nivel 2: Contenedores . . . 4
3.3 Modelo C4 – Nivel 3: Componentes (backend) . . . 4
3.4 Despliegue . . . 4

4 Flujo de trabajo (UML) **4**
4.1 Diagrama de secuencia . . . 4
4.2 Flujo de estados de la tarea . . . 5
4.3 Reglas de contención (obligatorio) . . . 5

5 Decisión de arquitectura (ADR-0001) **5**
5.1 Contexto . . . 5
5.2 Alternativas evaluadas . . . 5
5.3 Decisión . . . 5
5.4 Consecuencias . . . 6

6 Implementación **6**
6.1 Estructura del proyecto 6
6.2 Endpoints de la API 6
6.3 Tests 6

7 Guion de demo **6**
7.1 Objetivo 6
7.2 Guion paso a paso 7
7.3 Momento de contención (clave para el vídeo) 7
7.4 Requisitos técnicos para la demo 7

8 Próximos pasos **7**

9 Referencias **8**

1 Contexto del hackathon

El hackathon **Vultr: Agent Rush** es un evento híbrido organizado por lablab.ai y Vultr, con fase de construcción online del 3 al 8 de noviembre de 2026 y fase presencial el 8 de noviembre en Salt Lake City, UT.

1.1 Objetivo

Construir un agente de IA que realice trabajo real — escribir y ejecutar código o operar un navegador real — con cada acción contenida dentro de un sandbox aislado en Vultr. El sistema actúa como capa de control centralizada: planifica la tarea, la despacha a un entorno de ejecución aislado y devuelve salida verificable.

1.2 Desafío elegido: Blast Radius Zero

El agente web ejecuta trabajo real (código cuántico) dentro de un sandbox aislado en Vultr. Patrones de ejecución:

- **Pattern A (Sandboxed code execution):** usuario pide → modelo escribe código → código se ejecuta en contenedor → salida real retorna → modelo explica el resultado.
- **Pattern B (Sandboxed browser use):** objetivo → plan → Playwright en contenedor → capturas → visión verifica → aprobación humana.

Este proyecto se enfoca en el **Pattern A**, que es el núcleo del valor.

1.3 Stack tecnológico requerido

Componente	Detalle	Requerido
Vultr Cloud Compute	VM backend, capa de control	Sí (ambos)
Vultr Serverless Inference	OpenAI-compatible, `api.vultrinference.com/v1`	C1: Sí, C2: No
Agent sandboxing	Docker, OpenSandbox, gVisor, E2B, Microsandbox	C1: Sí
Vultr API	Instancias desechables por tarea	Opcional
GPUs de Vultr	No disponibles en este evento	—

Table 1: Stack tecnológico del hackathon

1.4 Entregables requeridos

1. Repositorio público de GitHub con setup y documentación.
2. Backend desplegado en VM de Vultr, LLM vía Vultr Serverless Inference.
3. URL pública de demo (experiencia con estilo de producto).
4. Vídeo de demo grabado incluyendo un momento de contención.
5. Explicación clara de la arquitectura y el caso de uso.
6. Envío oficial en lablab.ai.

1.5 Criterios de juzgamiento (1–5 cada uno)

1. Application of Technology
2. Presentation
3. Business Value
4. Originality

Advertencia: falta vídeo, enlace de demo o GitHub → calificación de 1 en Presentación. Tres de cuatro criterios se hunden por falta de paperwork.

2 Descripción del proyecto

2.1 Qué es

El **Quantum Testing Agent** es un agente web que recibe una especificación de circuito cuántico, planifica la prueba mediante un LLM (Vultr Serverless Inference), genera código en el framework elegido (Qiskit, PennyLane, Cirq, Azure Quantum), lo ejecuta dentro de un sandbox Docker aislado en una VM de Vultr y devuelve un resultado verificable.

2.2 Problema

Probar circuitos y simulaciones cuánticas es riesgoso: el código puede ser arbitrario, contener bucles infinitos, comandos destructivos o consumir recursos sin límite. Ejecutarlo directamente en el host o en el backend compromete la seguridad del sistema. Además, los frameworks cuánticos son heterogéneos (Qiskit/IBM, PennyLane/AWS, Cirq/Google, Azure Quantum/Microsoft), lo que complica la compatibilidad.

2.3 Solución

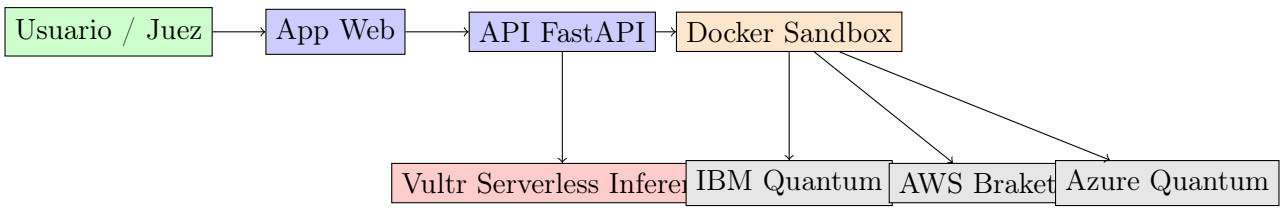
Un agente que **aisla cada ejecución** en un contenedor Docker desechable con límites de CPU, memoria y tiempo. El sandbox se destruye tras cada tarea, garantizando que ningún código cuántico toque el host ni el backend. La contención se demuestra en el vídeo de demo: el sandbox absorbe un bucle infinito o un comando `rm -rf` y el sistema permanece intacto.

2.4 Compatibilidad

- **Frameworks:** Qiskit (IBM), PennyLane (AWS Braket), Cirq (Google), Azure Quantum (Microsoft).
- **Hardware:** simulador CPU local como fallback (compatible con NVIDIA CUDA-Q y AMD ROCm); backends remotos opcionales para hardware cuántico real.
- **Nubes:** IBM Quantum, AWS Braket, Azure Quantum, Google Quantum.

3 Arquitectura

3.1 Modelo C4 – Nivel 1: Contexto



3.2 Modelo C4 – Nivel 2: Contenedores

Contenedor	Responsabilidad	Stack
App Web	UI de entrada y visualización	HTML/JS
API FastAPI	Endpoints REST: crear tarea, estado, resultado	Python, FastAPI, uvicorn
Agente Orquestador	Descompone tarea, genera código, verifica	Python, LLM vía Vultr
Docker Engine	Levanta/destruye contenedores aislados	Docker, cgroups
Sandbox por tarea	Ejecuta código cuántico con límites	python:3.11-slim

Table 2: Contenedores del sistema

3.3 Modelo C4 – Nivel 3: Componentes (backend)

Componente	Qué hace	Entrada	Salida
Task Planner	Descompone especificación en pasos	Especificación	Lista de pasos
Code Generator	Escribe código de prueba	Paso + framework	Archivo .py
Sandbox Runner	Levanta contenedor, límites, ejecuta, destruye	Archivo .py	Salida + código de
Result Verifier	Evalúa validez y contención	Salida cruda	Resultado verificab
Hardware Abstraction	Normaliza llamadas a backends	Resultado	Datos unificados

Table 3: Componentes del backend

3.4 Despliegue

- **VM Vultr:** backend FastAPI + Docker Engine como capa de control.
- **Vultr Serverless Inference:** llamadas LLM (planificación, generación, explicación).
- **Backends remotos:** IBM Quantum, AWS Braket, Azure Quantum (opcionales, configurables vía variables de entorno).
- **Simulador CPU local:** fallback sin dependencias externas.

4 Flujo de trabajo (UML)

4.1 Diagrama de secuencia

1. Usuario especifica circuito + framework + shots en la web.
2. Web envía `POST /tasks` a la API FastAPI.
3. API crea la tarea y llama al orquestador.
4. Orquestador pide plan de descomposición al LLM (Vultr Serverless Inference).
5. LLM devuelve pasos de prueba.
6. Orquestador genera código de prueba en el framework elegido.
7. Sandbox Runner levanta contenedor Docker aislado con límites.
8. Código cuántico se ejecuta dentro del sandbox.
9. Sandbox devuelve salida + código de salida.

10. Contenedor se destruye (disciplina de ciclo de vida).
11. Result Verifier verifica salida y contención.
12. Resultado verificable se devuelve a la UI web.

4.2 Flujo de estados de la tarea

`creada` → `planificada` → `codigo_generado` → `sandbox_levantado` → `ejecutando` → `completada` / `fallida_ejecucion` → `verificada` → `resultado_listo`

4.3 Reglas de contención (obligatorio)

1. **Aislamiento de procesos:** el código cuántico nunca toca el host ni el runtime de la API.
2. **Higiene de secretos:** ninguna API key dentro del sandbox; credenciales solo en el backend.
3. **Límites de recursos:** tope de CPU, memoria y tiempo por contenedor (ej. 60 s, 512 MB).
4. **Disciplina de ciclo de vida:** el contenedor se destruye tras cada tarea, exitoso o fallido.
5. **Momento de contención en demo:** el vídeo muestra el sandbox absorbiendo un `rm -rf`, bucle infinito o circuito hostil sin comprometer el sistema.

5 Decisión de arquitectura (ADR-0001)

5.1 Contexto

El proyecto participa en Vultr: Agent Rush (3–8 nov 2026). Desafío: Blast Radius Zero. Requisitos: VM Vultr como backend, Vultr Serverless Inference para LLM, Docker sandbox, sin GPUs disponibles.

5.2 Alternativas evaluadas

Alt.	Descripción	Ventajas	Desventajas
A1	Simulador CPU local	Sin dependencias externas	Limitado a CPU, no demuestra multi-nube
A2	Backends remotos (IBM/AWS/Azure)	Hardware cuántico real	Colas, costos, credenciales
A3	Híbrido: CPU fallback + remotos	Demo robusto + multi-nube	Mayor complejidad
A4	GPUs locales (NVIDIA/AMD)	Aceleración hardware	Inviable: sin GPUs en Vultr

Table 4: Alternativas de arquitectura

5.3 Decisión

Se adopta la **alternativa A3 (híbrida)**: simulador CPU local como fallback garantizado + adaptadores para IBM/AWS/Azure/Google como backends remotos opcionales. Toda la inferencia del agente usa Vultr Serverless Inference. El sandbox es un contenedor Docker desechable con límites, destruido tras cada tarea.

5.4 Consecuencias

- **Positivas:** demo robusto sin depender de colas externas; cumple requisitos del hackathon; contención verificable.
- **Negativas:** simulador CPU más lento que GPU; limitar demo a circuitos pequeños (≤ 20 qubits); complejidad de adaptadores.

6 Implementación

6.1 Estructura del proyecto

```
quantum-testing-agent/
 app/
    __init__.py
    main.py              # FastAPI: /health, /tasks, /tasks/{id}, /
    models.py            # CircuitSpec, TaskResult, TaskStatus, Framework
    store.py             # Almacenamiento en memoria
    orchestrator.py      # Planificación LLM + generación de código
    sandbox.py           # Docker sandbox con límites y destroy
    backend_adapters.py  # Qiskit, PennyLane, Cirq, Azure, CPU local
 tests/
    test_basic.py        # Tests de orquestador y store
    test_api.py          # Tests de API (health, tasks, contención)
 docs/
    C4.md
    workflow.md
    ADR/0001-*.md
    demo-script.md
 requirements.txt
 .env.example
 Dockerfile
 docker-compose.yml
 README.md
 AGENTS.md
```

6.2 Endpoints de la API

Método	Ruta	Descripción
GET	`/health`	Estado del servicio y sandbox
POST	`/tasks`	Crear y ejecutar tarea de prueba cuántica
GET	`/tasks/{id}`	Estado y resultado de la tarea
GET	`/`	UI web de demo

Table 5: Endpoints de la API

6.3 Tests

7 Guion de demo

7.1 Objetivo

Demostrar un agente web que prueba circuitos cuánticos de forma aislada y verificable en Vultr, cumpliendo el desafío Blast Radius Zero. Duración $\leq$ 5 minutos. Debe incluir un **momento de**

Test	Qué verifica
test_health	Endpoint /health responde OK
test_create_task	POST /tasks crea y ejecuta tarea
test_create_task_containment	Sandbox contiene código hostil
test_plan_task	Orquestador genera plan
test_generate_code	Genera código por framework
test_store_and_get	Almacenamiento y recuperación

Table 6: 6 tests, todos pasando

contención.

7.2 Guion paso a paso

#	Paso	Qué se muestra	Tiempo
1	Apertura	Problema: probar código cuántico es riesgoso; solución: sandbox aislado	30 s
2	Entrada	Usuario escribe circuito de 3 qubits + Hadamard + CNOT en Qiskit	30 s
3	Planificación	Agente (Vultr Inference) descompone en pasos, muestra plan	30 s
4	Generación	Código Qiskit generado, mostrado brevemente	20 s
5	Ejecución	Contenedor Docker levantado con límites, código ejecutado	45 s
6	Resultado	Estados de medida y histogramas verificables	30 s
7	Contención	Código hostil (bucle infinito) → sandbox lo absorbe y destruye	60 s
8	Multi-framework	Selección de PennyLane/Cirq, prueba repetida	30 s
9	Cierre	Resumen de arquitectura + llamada a acción	45 s

Table 7: Guion de demo (≈ 4 min 30 s)

7.3 Momento de contención (clave para el vídeo)

- **Entrada hostil:** código con `while True: pass` o `os.system("rm -rf /tmp/test")`.
- **Comportamiento:** el contenedor alcanza el límite de tiempo/memoria, Docker lo mata, el entorno se destruye, la API responde con `containment: true`.
- **Evidencia en vídeo:** terminal del backend registrando la destrucción del contenedor + web mostrando estado seguro.

7.4 Requisitos técnicos para la demo

8 Próximos pasos

1. Configurar `.env` con `VULTR_API_KEY` y `VULTR_INFERENCE_URL`.

Requisito	**Detalle**
Backend desplegado	VM de Vultr con FastAPI + Docker, URL pública
LLM	Vultr Serverless Inference
Frameworks en Docker	Qiskit, PennyLane (Cirq, Azure opcionales)
Límites de sandbox	60 s CPU, 512 MB memoria, red desactivada
Repo público	GitHub con README, setup, docs, guion de demo
Vídeo	$\leq$ 5 min, MP4, $\leq$ 300 MB, con contención

Table 8: Requisitos técnicos

2. Desplegar en VM de Vultr: `docker-compose up -d`.
3. Grabar vídeo de demo ($\leq$ 5 min) con momento de contención.
4. Subir repo a GitHub público y enviar en lablab.ai.
5. Verificar que la URL de demo es accesible públicamente.

9 Referencias

- Vultr: Agent Rush Hackathon — `https://lablab.ai/ai-hackathons/vultr-hackathon`
- Vultr Community — `https://community.vultr.com/e/vultr-agent-rush-hackathon`
- Luma — `https://luma.com/lablab-8d5a`
- Repositorio GitHub — `https://github.com/jalp17/quantum-testing-agent`